

Contents

 OpenCV 三周入门手册	1
一、教学目标	2
学习策略: Python 先行, C++ 优化	2
推荐学习资料	2
二、教程安排	3
三、预期成果	4
四、项目结构准备	4
操作步骤	4
文件组织规范	5
提交至 Git 与 GitHub	5
五、环境配置	5
Day 0: 环境搭建	5
六、每日学习内容	6
第一周: 图像基础与 OpenCV 入门	6
Day 1: OpenCV 简介与图像读写	6
Day 2: 颜色空间与通道操作	8
Day 3: 图像缩放、ROI 与几何变换	9
Day 4: 阈值分割	10
Day 5: 形态学操作	11
Day 6: 图像滤波与降噪	13
Day 7: 视频处理与综合实验	14
第一周小结	15
第二周: 边缘检测与特征提取	16
Day 8: 边缘检测	16
Day 9: 轮廓检测与几何分析	17
Day 10-11: HOG 特征与 SVM 分类	18
Day 12-13: 颜色识别与目标追踪	22
Day 14: 第二周复习与综合练习	23
第三周: 局部特征、相机定位与视觉标签	25

OpenCV 三周入门手册

version: 1.1.0 状态: 第二周完成, 第三周编写中.....

一、教学目标

本手册旨在帮助已掌握 Python 和 C++ 基础的同学，在三周时间内比较系统掌握 OpenCV 的核心功能与计算机视觉基础知识。

希望学习完成后，同学们能够：

- 理解图像处理与计算机视觉的基础概念（颜色空间、图像矩阵、滤波、形态学等）
- 熟练使用 OpenCV 进行图像/视频的读写、显示、处理
- 掌握边缘检测、轮廓分析、特征提取与匹配等核心算法
- 使用相机标定、PnP、视觉标签实现目标定位
- 完成综合大作业，将所学知识应用于实践

【注】本手册**不能**替代系统的计算机视觉课程，仅作为 OpenCV 入门的快速指南。建议结合推荐资料深入学习。

学习策略：Python 先行，C++ 优化

本手册采用 “Python 原型验证 + C++ 性能部署” 的双语言策略：

- **Python**：语法简洁，适合快速验证算法、熟悉 API、开发原型
- **C++**：性能更高，适合部署到比赛机器人等实时性要求高的场景

建议学习路径：先用 Python 跑通算法流程，再将关键模块迁移到 C++ 进行优化。

推荐学习资料

【注】在之后的内容中 『』 代表可点击的链接

更系统的学习资料请参考以下资源：

在线教程：

- 『OpenCV 官方中文教程 (4.x) 』：官方文档的中文翻译，内容权威全面，**强烈推荐！**
- 『菜鸟教程 - OpenCV 教程』：提供 Python 和 C++ 两个版本的简明教程，适合快速入门。
- 『OpenCV-Python 教程介绍』：一个较为详细的 OpenCV-Python 教程，涵盖基础和进阶内容。
- 『OpenCV 官方文档 (英文) 』：最新最全的官方参考，适合查阅 API。

书籍推荐：

- 《Learning OpenCV 4》：适合初学者和进阶，内容涵盖 OpenCV 4 的核心功能。可选 python 和 C++ 版本。中文版名为《学习 OpenCV 4》。

还可以自行搜索相关主题的博客和视频教程，如 Bilibili、YouTube 上的 OpenCV 教学视频，或 Medium、CSDN 等平台的技术文章。

以上资源均适合有一定编程基础的学习者。建议选择一个**能接受学进去的**作为主要材料，其他作为辅助参考。

学习 OpenCV 主要以掌握**核心概念**和**常用 API** 为主，建议多动手实践，边学边做。各种算法的**数学原理**和**细节**可根据兴趣深入研究，但**不必过于纠结于理论推导**。

二、教程安排

教程共 **21 天**，分为三周进行。

周次	主题	核心内容	综合实验
第 1 周	图像基础 + OpenCV 入门 + 阈值化 & 形态学	像素/通道/颜色空间、读写显示、阈值分割、形态学操作、滤波降噪	视频转字符画
第 2 周	边缘检测 + 特征提取 + SVM 分类 + 颜色识别	Sobel/Canny 边缘检测、轮廓分析、HOG+SVM 分类、HSV 颜色识别	手写数字识别 & 颜色追踪
第 3 周	局部特征 + 相机定位 + 视觉标签	角点检测、SIFT/ORB 特征、特征匹配、相机标定、PnP、双目视觉、AprilTag	目标定位系统

本教程每天的学习内容包括：

- 📖 **知识点**：当天需要掌握的核心概念与 API
- 🔗 **学习资料**：推荐的学习资源链接
- ✅ **应知应会 Checklist**：当天学习结束后需要掌握的技能点
- 📝 **小任务**：通过编写代码完成的小练习，巩固当天所学内容

【注】本教程提供的小任务和大作业均为参考示例，学习者可根据自身兴趣进行调整。**不强制要求完成**，完成情况**不计入最终考核评分**。

三、预期成果

- 同学们能独立编写 OpenCV 程序处理图像/视频
 - 能熟练使用阈值、形态学、边缘检测、轮廓分析等基础算法
 - 理解特征提取与匹配的原理，能实现简单的目标识别
 - 掌握相机标定与 PnP 位姿估计，能进行目标定位
 - 完成一个完整的“目标定位系统”大作业
-

四、项目结构准备

在开始正式学习之前，我们需要先准备好项目的基本目录结构，方便每天的代码和总结文档进行管理。

操作步骤

1. 打开你的工作目录（例如 D:\my-cv-project 或 ~/my-cv-project）
2. 在其中新建一个文件夹：

```
my-cv-project/  
opencv学习/
```

3. 在 opencv 学习 文件夹下，建立 Week1 ~ Week3 共 3 个子文件夹，每周文件夹下再建立 Day 子文件夹：

```
opencv学习/  
Week1/  
    Day1/  
    Day2/  
    ...  
    Day7/  
Week2/  
    Day8/  
    ...  
    Day14/  
Week3/  
    Day15/  
    ...  
    Day21/
```

文件组织规范

每个 DayX/ 文件夹下，至少包含：

- 当天的代码文件 (Python: task.py, C++: task.cpp)
- 如果有总结或笔记，推荐使用 **Markdown 文件** (例如 notes.md)

例如，Day1/ 文件夹可能长这样：

Day1/

task.py	# Python** 小任务**代码
task.cpp	# C++** 小任务**代码 (可选)
data/	# 测试图片/视频
notes.md	*** 知识点**总结

提交至 Git 与 GitHub

```
cd my-cv-project
git init
git add .
git commit -m "完成 DayX 学习任务"
git remote add origin <你的远程仓库地址>
git push -u origin main
```

【注】推荐将每日的进度提交到 Git 仓库，养成良好的版本管理习惯。

五、环境配置

Day 0: 环境搭建

0.1 基本知识点

- 安装 Python 3.x 与 OpenCV
- 配置 C++ OpenCV 开发环境 (可选)
- 验证安装是否成功

0.2 学习资料

- 『菜鸟教程 - OpenCV 安装』
- 『OpenCV 官方安装指南』
- C++ 环境配置 (基于 CMake) 可参考

- 博客：『Vscode+Cmake 配置并运行 opencv 环境 (Windows 和 Ubuntu 大同小异)』

大体上来说，Python 环境安装较为简单，推荐优先使用 Python 进行学习和实验。C++ 环境配置相对复杂，初学者可根据需要选择性完成。【但请务必保证最终能够顺利使用 C++ 进行开发，最终考核要求使用 C++ 进行开发】

0.3 应知应会 Checklist

- ☐ 会使用 conda/pip 安装 opencv-python 和 opencv-contrib-python
- ☐ 能在 Python 中 import cv2 并打印版本号
- ☐ 【初学时可选】能在 C++ 中编译运行 OpenCV 程序

0.4 小任务 Python 环境安装：

创建虚拟环境 (可选)

```
conda create -n opencv_env python=3.10 -y
conda activate opencv_env
```

安装 OpenCV 及常用库

```
pip install opencv-python opencv-contrib-python numpy matplotlib
```

验证安装：

```
import cv2
print(f"OpenCV 版本: {cv2.__version__}")
```

C++ 环境配置请参考上述博客链接，根据你的操作系统和 IDE 进行相应设置。

六、每日学习内容

第一周：图像基础与 OpenCV 入门

Day 1: OpenCV 简介与图像读写

1.1 基本知识点

- 了解 OpenCV 的基本模块与功能 (『OpenCV 基础模块』)
- 感受 Python 与 C++ 接口差异
- OpenCV 图像表示: NumPy 数组与 cv::Mat (『cv::Mat - 基本图像容器』)【了解即可，可在后续的使用中加深理解】
- BGR 与 RGB 的默认约定 (『【超直白讲解 opencv RGB 与 BGR】RGB 模式与 BGR 模式有什么不同，如何相互转换?』)

- 图像读取、显示、保存
- 【扩展-可选】了解 OpenCV Core 模块的常用数据类型与操作 (『OpenCV Core 模块』)

1.2 学习资料

- 『菜鸟教程 - OpenCV 入门实例』
- 『OpenCV 中文教程 - 图像入门』

1.3 应知应会 Checklist

- ☐ 理解 OpenCV 的基本模块结构
- ☐ 1 了解图像在内存中的表示形式
- ☐ 能使用 `cv.imread()` 读取图片
- ☐ 能使用 `cv.imshow()` 显示图片
- ☐ 能使用 `cv.imwrite()` 保存图片
- ☐ 理解 BGR 与 RGB 的区别
- ☐ 【扩展-可选】了解 OpenCV Core 模块的常用数据类型与操作

1.4 小任务 读取一张图片并显示，然后保存为另一个文件。

Python 示例:

```
import cv2 as cv

# 读取图片
img = cv.imread('data/cat.jpg')
if img is None:
    print('无法读取图片')
    exit()

# 显示图片
cv.imshow('My Image', img)
cv.waitKey(0)
cv.destroyAllWindows()

# 保存图片
cv.imwrite('output/cat_copy.png', img)
```

C++ 示例:

```
#include <opencv2/opencv.hpp>
using namespace cv;
```

```
int main() {
    Mat img = imread("data/cat.jpg");
    if (img.empty()) return -1;
    imshow("My Image", img);
    waitKey(0);
    imwrite("output/cat_copy.png", img);
    return 0;
}
```

Day 2: 颜色空间与通道操作

2.1 基本知识点

- BGR、RGB、GRAY、HSV 颜色空间
- 颜色空间转换: `cvtColor()`
- 通道分离与合并: `split()` / `merge()`
- 像素访问与修改
- 基于颜色空间的简单颜色分割: `inRange()`

2.2 学习资料

- 『OpenCV 中文教程 - 颜色空间转换』
- 『OpenCV 中文教程 - 使用 `inRange` 的阈值操作』
- 相关博客:
 - 『OpenCV 颜色空间详解及转换』

2.3 应知应会 Checklist

- ☐ 能将 BGR 转换为 GRAY、HSV
- ☐ 能分离和合并图像通道
- ☐ 理解 HSV 颜色空间的含义 (H: 色调, S: 饱和度, V: 明度)

2.4 小任务 (1) 在搜索引擎中查找并了解 BGR、RGB、HSV 颜色空间的区别与应用场景。
(2) 用 HSV 颜色空间实现物体颜色分割。

```
import cv2 as cv

img = cv.imread('data/ball.jpg') # 包含绿色球的图片, 也可根据需要更换
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
```

```
# 绿色范围 (根据实际调整)
low = (35, 80, 40)
high = (85, 255, 255)

mask = cv.inRange(hsv, low, high)
result = cv.bitwise_and(img, img, mask=mask)

cv.imshow('Original', img)
cv.imshow('Mask', mask)
cv.imshow('Result', result)
cv.waitKey(0)
```

Day 3: 图像缩放、ROI 与几何变换

3.1 基本知识点

- 图像缩放: `resize()`
- 感兴趣区域 (ROI): 数组切片 (Python) / `cv::Rect(C++)`
- 几何变换: `remap()`
- 仿射变换: `warpAffine()`
- 透视变换: `warpPerspective()`

3.2 学习资料

- 『菜鸟教程 - 图像基本操作』
- 『OpenCV 中文教程 - 几何变换』
- 『OpenCV 中文教程 - ReMap』

3.3 应知应会 Checklist

- ☐ 能使用 `resize()` 缩放图像
- ☐ 能通过数组切片获取 ROI
- ☐ 能使用 `remap()` 实现复杂几何变换
- ☐ 能实现图像旋转和透视变换

3.4 小任务 提取图像的 ROI 并缩放显示。

```
import cv2 as cv
```

```

img = cv.imread('data/cat.jpg')

# 提取 ROI (y1:y2, x1:x2)
roi = img[50:300, 100:400]

# 缩放
small = cv.resize(img, (320, 240), interpolation=cv.INTER_LINEAR)
large = cv.resize(roi, None, fx=2, fy=2)

cv.imshow('ROI', roi)
cv.imshow('Small', small)
cv.imshow('Large ROI', large)
cv.waitKey(0)

```

Remap 和更复杂的几何变换可在后续学习中深入了解。其与 resize 的区别在于：

- resize = 只能做缩放 → 固定规则转换
- remap = 完整几何变换引擎 → 任意映射、畸变校正、warp 所有操作都靠它

Day 4: 阈值分割

4.1 基本知识点

- 全局阈值: threshold()
- 自适应阈值: adaptiveThreshold()
- OTSU 自动阈值
- 阈值类型: BINARY、BINARY_INV、TRUNC、TOZERO 等
- 【扩展-可选】更多分割方式：如“图像分割与距离变换和分水岭算法”、“通过梯度结构张量进行各向异性图像分割”。

4.2 学习资料

- 『菜鸟教程 - 图像阈值处理』
- 『OpenCV 中文教程 - 图像阈值』
- 博客: 『OpenCV-图像二值化 (Binary Image)』
- 【扩展-可选】『OpenCV 中文教程 - 图像分割与距离变换和分水岭算法』
- 【扩展-可选】『通过梯度结构张量进行各向异性图像分割』

4.3 应知应会 Checklist

- ☐ 对比基于 `inRange()` 的颜色阈值分割
- ☐ 能使用全局阈值进行二值化
- ☐ 能使用自适应阈值处理光照不均的图像
- ☐ 理解 OTSU 方法的原理和使用场景
- ☐ 【扩展-可选】了解距离变换和分水岭算法的基本概念
- ☐ 【扩展-可选】了解梯度结构张量在图像分割中的应用

4.4 小任务 比较三种阈值方法的效果。

```
import cv2 as cv

img = cv.imread('data/text.jpg')
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# 全局阈值
_, th1 = cv.threshold(gray, 127, 255, cv.THRESH_BINARY)

# OTSU 自动阈值
_, th2 = cv.threshold(gray, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

# 自适应阈值
th3 = cv.adaptiveThreshold(gray, 255, cv.ADAPTIVE_THRESH_GAUSSIAN_C,
                           cv.THRESH_BINARY, 11, 2)

cv.imshow('Global', th1)
cv.imshow('OTSU', th2)
cv.imshow('Adaptive', th3)
cv.waitKey(0)
```

除了上述三种阈值方法外，OpenCV 还提供其他的阈值处理方式，如基于三角形法的自动阈值等，感兴趣的同学可以自行查阅相关资料进行学习。

Day 5: 形态学操作

5.1 基本知识点

- 腐蚀: `erode()` 消除噪点、细化边界
- 膨胀: `dilate()` 填充空洞、加粗边界

- 开运算：先腐蚀后膨胀 去除小噪点
- 闭运算：先膨胀后腐蚀 填充小空洞
- 结构元素：getStructuringElement()

5.2 学习资料

- 『菜鸟教程 - 图像形态学操作』
- 『OpenCV 中文教程 - 形态学变换』
- 『OpenCV 中文教程 - 腐蚀与膨胀』
- 『OpenCV 中文教程 - 更多形态学变换』

5.3 应知应会 Checklist

- ☐ 能使用腐蚀和膨胀操作
- ☐ 能使用开闭运算去噪和填洞
- ☐ 理解结构元素的形状和大小对结果的影响

5.4 小任务 对二值图像进行形态学去噪。

```
import cv2 as cv

img = cv.imread('data/noisy_binary.png', 0)
kernel = cv.getStructuringElement(cv.MORPH_RECT, (5, 5))

# 开运算去除噪点
opened = cv.morphologyEx(img, cv.MORPH_OPEN, kernel)

# 闭运算填充空洞
closed = cv.morphologyEx(img, cv.MORPH_CLOSE, kernel)

cv.imshow('Original', img)
cv.imshow('Opened', opened)
cv.imshow('Closed', closed)
cv.waitKey(0)
```

可以尝试不同的卷积核形状（矩形、椭圆、十字形）和大小，交替使用膨胀和腐蚀操作并交换其顺序，观察对结果的影响。

Day 6: 图像滤波与降噪

6.1 基本知识点

- 均值滤波: `blur()` 简单平滑
- 高斯滤波: `GaussianBlur()` 适合高斯噪声
- 中值滤波: `medianBlur()` 适合椒盐噪声
- 双边滤波: `bilateralFilter()` 保边平滑

6.2 学习资料

- 『菜鸟教程 - 图像平滑处理』
- 『OpenCV 中文教程 - 图像平滑 1』
- 『OpenCV 中文教程 - 图像平滑 2』
- 【扩展-可选】『离焦模糊滤波器』
- 【扩展-可选】『运动去模糊滤波器』
- 【扩展-可选】『周期性噪声消除滤波器』

6.3 应知应会 Checklist

- ☐ 能根据噪声类型选择合适的滤波器
- ☐ 理解各种滤波器的优缺点
- ☐ 能调整滤波器参数以获得最佳效果
- ☐ 【扩展-可选】了解离焦模糊、运动去模糊和周期性噪声消除滤波器的基本原理，使用场景及实现方法

6.4 小任务 比较不同滤波器对椒盐噪声和高斯噪声的处理效果。

```
import cv2 as cv

img = cv.imread('data/noisy.jpg')

blur = cv.blur(img, (5, 5))
gblur = cv.GaussianBlur(img, (5, 5), 0)
mblur = cv.medianBlur(img, 5)
bblur = cv.bilateralFilter(img, 9, 75, 75)

cv.imshow('Original', img)
cv.imshow('Mean', blur)
cv.imshow('Gaussian', gblur)
cv.imshow('Median', mblur)
```

```
cv.imshow('Bilateral', bblur)
cv.waitKey(0)
```

Day 7: 视频处理与综合实验

7.1 基本知识点

- 视频读取: VideoCapture()
- 视频写入: VideoWriter()
- 帧率控制与键盘事件
- 摄像头实时处理

7.2 学习资料

- 『菜鸟教程 - OpenCV 视频处理 - 1』
- 『菜鸟教程 - OpenCV 视频处理 - 2』
- 『OpenCV 中文教程 - 视频入门』

7.3 应知应会 Checklist

- ☐ 能读取视频文件和摄像头
- ☐ 能逐帧处理视频并显示
- ☐ 能将处理后的视频保存到文件
- ☐ 能使用 waitKey() 控制帧率和响应按键

7.4 小任务: 视频转字符画 实现一个视频转字符画的程序。

```
import cv2 as cv
import time

# 读取 mp4 视频文件, 请将路径替换为你的视频文件路径
cap = cv.VideoCapture('./image//bad_apple.mp4')
chars = [' ', '#'] # 二值化: 空格表示白, # 表示黑

while True:
    ret, frame = cap.read()
    if not ret or frame is None:
        break
```

```

# 转灰度并缩小
if frame.ndim == 3:
    gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
else:
    gray = frame
small = cv.resize(gray, (120, 40))

# 转字符画
lines = []
for row in small:
    line = ''.join('#' if pixel > 127 else ' ' for pixel in row)
    lines.append(line)
ascii_art = '\n'.join(lines)

# 清屏并打印 (Windows: cls, Linux/Mac: clear)
print('\033[H\033[J' + ascii_art)

time.sleep(0.03) # 控制播放速度, 约 30fps

cap.release()

```

第一周小结

第一周 Checklist

- ☐ 能读取、显示、保存图像/视频
- ☐ 能完成基本颜色空间转换和掩膜分割
- ☐ 掌握阈值与形态学的基础概念并能去噪
- ☐ 能搭建基本的实时视频处理管线

Python C++ 移植提示

1. OpenCV 在 Python 和 C++ 中都使用 BGR 颜色顺序
2. Python 的 numpy 数组对应 C++ 的 cv::Mat, 注意数据类型匹配
3. C++ 处理大量帧时通常比 Python 快 3-10 倍
4. 函数名基本一致 (如 cvtColor、threshold), 但参数形式略有不同

第一周综合任务: C++ 实现视频转字符画 要求:

- 使用 C++ 实现 Day 7 的视频转字符画功能
- 优化性能，保证流畅显示
- 通过形态学操作、滤波等方法提升字符画质量

Day 7 的实现中仅支持使用两个字符（空格和 #），C++ 版本可尝试使用更多字符实现更丰富的灰度表现。例如，使用 10 个字符：[' ', '.', ':', '-', '=', '+', '*', '#', '%', '@'] 代表十个灰度等级。

第二周：边缘检测与特征提取

Day 8：边缘检测

8.1 基本知识点

- Sobel 算子：计算图像梯度
- Laplacian 算子：二阶导数
- Canny 边缘检测：多阶段算法，效果最好

8.2 学习资料

- 『菜鸟教程 - 边缘检测』
- 『OpenCV 中文教程 - Canny 边缘检测』
- 『OpenCV 中文教程 - Sobel 导数』
- 『OpenCV 中文教程 - Laplacian 算子』
- 『OpenCV 中文教程 - Canny 边缘检测』

8.3 应知应会 Checklist

- ☐ 能使用 Sobel 计算 x/y 方向梯度
- ☐ 能使用 Canny 进行边缘检测
- ☐ 理解 Canny 的双阈值原理

8.4 小任务 对图像进行 Canny 边缘检测并显示结果。

```
import cv2 as cv

img = cv.imread('data/building.jpg')
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# Sobel
sobelx = cv.Sobel(gray, cv.CV_64F, 1, 0, ksize=3)
```

```
sobely = cv.Sobel(gray, cv.CV_64F, 0, 1, ksize=3)

# Canny
edges = cv.Canny(gray, 50, 150)

cv.imshow('Sobel X', cv.convertScaleAbs(sobelx))
cv.imshow('Sobel Y', cv.convertScaleAbs(sobely))
cv.imshow('Canny', edges)
cv.waitKey(0)
```

Day 9: 轮廓检测与几何分析

9.1 基本知识点

- 轮廓查找: `findContours()`
- 轮廓绘制: `drawContours()`
- 轮廓属性: 面积、周长、边界框、最小外接矩形/圆
- 轮廓近似: `approxPolyDP()` 多边形逼近
- 凸包: `convexHull()`
- 霍夫变换 (Hough Transform): 直线检测与圆检测

9.2 学习资料

- 『菜鸟教程 - 轮廓检测』
- 『OpenCV 中文教程 - 轮廓入门』
- 『OpenCV 中文教程 - 轮廓检测』
- 『OpenCV 中文教程 - 凸包』
- 『OpenCV 中文教程 - 边界矩形与最小外接圆』
- 『OpenCV 中文教程 - 旋转矩形与椭圆』
- 『OpenCV 中文教程 - 霍夫变换 (直线)』
- 『OpenCV 中文教程 - 霍夫变换 (圆)』

9.3 应知应会 Checklist

- ☐ 能使用 `findContours()` 提取轮廓
- ☐ 能计算轮廓的面积、周长、边界框
- ☐ 能使用多边形逼近识别形状 (三角形、矩形、圆形)
- ☐ 能使用霍夫变换检测直线和圆

9.4 小任务 检测图像中的几何形状并标注。

```
import cv2 as cv

img = cv.imread('data/shapes.png')
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
edges = cv.Canny(gray, 50, 150)
contours, _ = cv.findContours(edges, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)

for cnt in contours:
    area = cv.contourArea(cnt)
    if area < 200:
        continue

    # 多边形逼近
    epsilon = 0.02 * cv.arcLength(cnt, True)
    approx = cv.approxPolyDP(cnt, epsilon, True)

    # 判断形状
    if len(approx) == 3:
        label = 'Triangle'
    elif len(approx) == 4:
        label = 'Rectangle'
    else:
        label = 'Circle'

    # 绘制边界框和标签
    x, y, w, h = cv.boundingRect(cnt)
    cv.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv.putText(img, label, (x, y-10), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0),

cv.imshow('Shapes', img)
cv.waitKey(0)
```

Day 10-11: HOG 特征与 SVM 分类

10.1 基本知识点

HOG + SVM 是经典的目标检测方法，常用于行人检测等任务。其基本流程为：

1. 收集正负样本图像
 2. 提取 HOG 特征
 3. 使用 SVM 训练分类器
 4. 保存训练好的模型
 5. 在新图像上提取 HOG 特征并使用 SVM 进行分类
- **HOG** (方向梯度直方图) 特征: 描述局部梯度方向分布
 - **SVM** (支持向量机) 分类器
 - 训练流程: 特征提取 训练 保存模型 推理
 - **scikit-learn** 是一个流行的机器学习库, 提供了方便的常用机器学习算法实现, 适合快速实验和原型开发。
 - **scikit-learn** 提供了方便的 SVM 实现, 但其为 python 生态, 与 C++ 兼容性较差。OpenCV 也提供了 SVM 实现, 推荐在 C++ 中使用 OpenCV 的 SVM 模块。
 - 除了 SVM, 还有其他分类器可选, 如**决策树**、**随机森林**、**神经网络**等, 但 SVM 是经典且易于实现的入门选择。这些都是**机器学习**的基础内容, 感兴趣的同学可以自行查阅相关资料进行学习。

10.2 学习资料

- 博客:『一文讲解方向梯度直方图 (hog)』
- 『scikit-learn SVM 文档』
- 『OpenCV 中文教程 - SVM』
- 『OpenCV 中文教程 - 非线性 SVM』
- 『HOG + SVM 进行分类的基本流程』
- 案例:『使用 HOG + SVM 实现手写数字分类』

10.3 应知应会 Checklist

- ☐ 理解 HOG 特征的原理
- ☐ 能使用 HOG + SVM 实现简单分类器
- ☐ 能保存和加载训练好的模型

10.4 小任务 使用 HOG + SVM 实现手写数字分类。

```
import cv2 as cv
import numpy as np
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_digits
from skimage.feature import hog
```

```

from joblib import dump, load

# 加载数据, 这里使用 sklearn 自带的手写数字数据集
digits = load_digits()
X, y = digits.images, digits.target

# 提取 HOG 特征
X_hog = []
for img in X:
    img_uint8 = (img * 16).astype(np.uint8)
    hog = cv.HOGDescriptor((8, 8), (4, 4), (4, 4), (4, 4), 9)
    fd = hog.compute(img_uint8)
    X_hog.append(fd.flatten())
X_hog = np.array(X_hog)

#==== 使用 scikit-learn 的 SVM 实现 ====
# 训练 ——同时保留原始图像用于可视化
X_train, X_test, imgs_train, imgs_test, y_train, y_test = train_test_split(
    X_hog, X, y, test_size=0.2, random_state=42
)
clf = svm.SVC(kernel='linear')
clf.fit(X_train, y_train)
print(f'准确率: {clf.score(X_test, y_test):.2%}')

# 用原始图像可视化部分测试样本及其预测结果
for i in range(5):
    # imgs_test 存储原始 8x8 浮点图像 (0..16), 先还原为 uint8 后放大显示
    img = (imgs_test[i] * 16).astype(np.uint8)
    img = cv.resize(img, (200, 200), interpolation=cv.INTER_NEAREST)
    pred = clf.predict([X_test[i]])[0]
    true = y_test[i]
    # 图中添加预测结果文字
    img = cv.cvtColor(img, cv.COLOR_GRAY2BGR)
    cv.putText(img, f'True: {true}', (10, 30), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0,
    cv.putText(img, f'sklearn Pred: {pred}', (10, 70), cv.FONT_HERSHEY_SIMPLEX,
    cv.imshow(f'sklearn SVM - True: {true} Pred: {pred}', img)
    cv.waitKey(0)
cv.destroyAllWindows()

```

```

# 保存模型
dump(clf, 'hog_svm.joblib')

# ===== 使用 opencv-python 的 SVM 实现 (与 scikit-learn 的实现类似)
# 准备数据 (OpenCV 要求 float32 的样本矩阵和 int32 的响应)
X_train_cv = X_train.astype(np.float32)
X_test_cv = X_test.astype(np.float32)
y_train_cv = y_train.astype(np.int32)
y_test_cv = y_test.astype(np.int32)

svm_cv = cv.ml.SVM_create()
svm_cv.setType(cv.ml.SVM_C_SVC)
svm_cv.setKernel(cv.ml.SVM_LINEAR)
svm_cv.setTermCriteria((cv.TERM_CRITERIA_MAX_ITER, 1000, 1e-6))

# 训练
train_data = cv.ml.TrainData_create(X_train_cv, cv.ml.ROW_SAMPLE, y_train_cv)
svm_cv.train(train_data)

# 评估准确率
_, resp = svm_cv.predict(X_test_cv)
preds_cv = resp.flatten().astype(np.int32)
acc_cv = (preds_cv == y_test_cv).mean()
print(f'OpenCV SVM 准确率: {acc_cv:.2%}')

# 可视化部分测试样本的预测 (来自 OpenCV SVM)
for i in range(5):
    img = (imgs_test[i] * 16).astype(np.uint8)
    img = cv.resize(img, (200, 200), interpolation=cv.INTER_NEAREST)
    img = cv.cvtColor(img, cv.COLOR_GRAY2BGR)
    pred_opencv = int(preds_cv[i])
    true = int(y_test_cv[i])
    cv.putText(img, f'True: {true}', (10, 30), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255))
    cv.putText(img, f'OpenCV Pred: {pred_opencv}', (10, 70), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0))
    cv.imshow(f'OpenCV SVM - True:{true} Pred:{pred_opencv}', img)
    key = cv.waitKey(0)
    if key == 27: # 按 Esc 可提前退出
        break

```

```
cv.destroyAllWindows()

# 保存 OpenCV SVM 模型
svm_cv.save('hog_svm_opencv.yml')
```

Day 12-13: 颜色识别与目标追踪

12.1 基本知识点 目标追踪 (Object Tracking) 的核心任务是：在时间序列图像中，对每一帧产生的目标检测结果进行时序关联 (temporal association)，从而为同一物理实体在不同时间上的观测分配一致的身份标识 (ID)。

目标追踪的基本流程为：

1. 通过各种检测方法 (颜色、形状、预训练模型等) 获取目标位置
 2. 计算目标的质心或边界框
 3. 使用各种追踪算法 (简单质心追踪、卡尔曼滤波、SORT、Deep SORT、OpenCV 内置追踪器等) 进行连续帧的目标位置预测与更新
- HSV 颜色阈值分割
 - 连通域分析: `connectedComponentsWithStats()` (对于使用阈值分割后的二值图像可快速提取目标区域)
 - 质心计算与追踪 (例如使用图像矩、用轮廓的中心)
 - 目标跟踪是一个复杂的进阶主题，以下仅介绍几种基础方法：
 - 简单质心追踪 (在连续帧中寻找最近质心)
 - 使用 OpenCV 内置追踪器 (如 KCF、MIL、CSRT 等)
 - 更多进阶方法如卡尔曼滤波、SORT、Deep SORT 等感兴趣可自行查阅相关资料学习

12.2 学习资料

- 『OpenCV 中文教程 - 颜色空间』
- 博客: 『OpenCV 笔记 (13): 连通域分析』
- 『OpenCV 中文教程 - 图像矩』
- 『OpenCV 中文教程 - 目标追踪』

12.3 应知应会 Checklist

- ☐ 能使用 HSV 进行颜色分割
- ☐ 能使用连通域分析提取目标
- ☐ 能计算目标质心并实现简单追踪

12.4 小任务 实现颜色目标追踪程序。

```
import cv2 as cv

cap = cv.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv = cv.cvtColor(frame, cv.COLOR_BGR2HSV)

    # 蓝色范围 (可调整)
    mask = cv.inRange(hsv, (100, 80, 40), (130, 255, 255))

    # 连通域分析
    num, labels, stats, centroids = cv.connectedComponentsWithStats(mask)

    for i in range(1, num): # 跳过背景
        x, y, w, h, area = stats[i]
        cx, cy = centroids[i]
        if area > 500: # 过滤小区域
            cv.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv.circle(frame, (int(cx), int(cy)), 5, (0, 0, 255), -1)

    cv.imshow('Tracking', frame)
    if cv.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv.destroyAllWindows()
```

Day 14: 第二周复习与综合练习

综合任务

1. **手写数字识别**: 完善 HOG+SVM 模型, 尝试在自己的手写数字上测试
2. **颜色追踪**: 实现多颜色目标追踪, 显示运动轨迹

3. **边缘与轮廓分析**：对复杂图像进行边缘检测与轮廓分析，识别多种形状
4. **综合应用**：结合边缘检测、轮廓分析与颜色追踪，实现一个简单的视觉特征检测与追踪系统，以下为示意图：

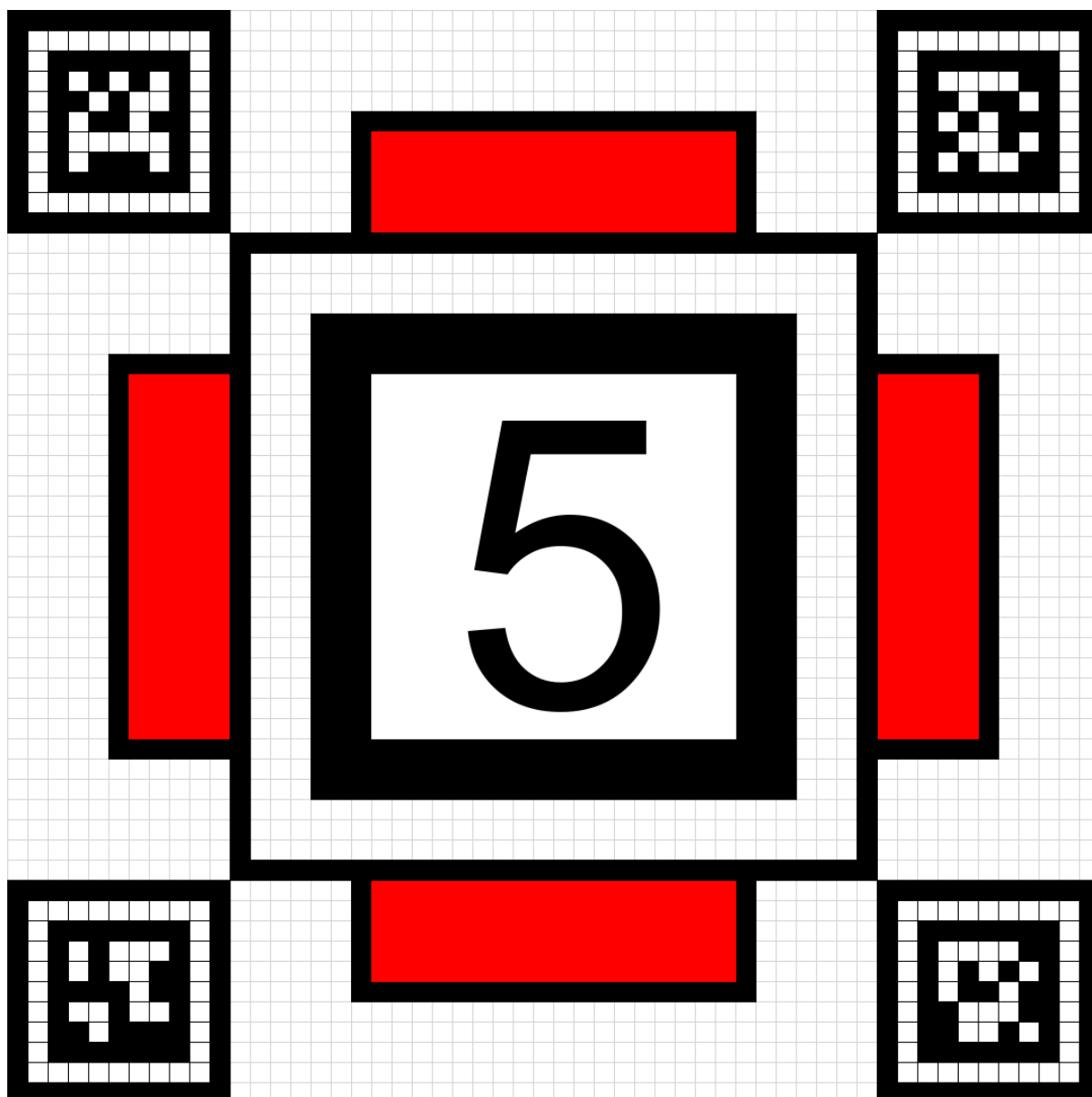


Figure 1: 测试图片

Tips:

1. 可以将该视觉特征打印出来，放置在摄像头前进行测试和调试。
2. 该视觉特征的四个角分别为 0-3 的 Apriltag 标签 (tag25h9 家族)，OpenCV 提供了对 AprilTag 的支持，感兴趣的同学可以查阅相关资料进行学习和实现。
3. 可以直接通过 AprilTag 标签得到 ROI 区域，然后在该区域内进行颜色分割和轮廓分析，从而实现对标签的颜色特征检测与追踪。在下一周我们将

介绍其他局部特征检测与描述子方法，避免对 AprilTag 产生依赖。

4. 除了简单的基于 HSV 颜色阈值分割外，还可以尝试：

- 使用 K-Means 聚类等方法进行颜色分割，从而提升鲁棒性。
 - 通过训练一个简单的分类器（如 SVM、决策树、随机森林等）来对颜色进行分类，从而提升识别准确率。
-

第三周：局部特征、相机定位与视觉标签

第三周内容正在编写中，敬请期待！